DAY-06 · LLM FINE-TUNING THEORY

Day-06:
LLM 파인튜닝 이론

적은 데이터와 적은 자원으로 도메인 성능을
끌어올리는 실전 파인튜닝

- Transformer 구조와 Attention 이해
- KV-Cache · Auto-Regressive 생성 원리
- SFT(Supervised Fine-Tuning) 실전 전략
- Embedding Tuning과 Contrastive Learning

LLM 기본 구조: Decoder-only Transformer

Tokenizer
텍스트 → 토큰 ID

→

Embedding
토큰 → 벡터

→

× N layers
Transformer Block
Self-Attention + FFN

→

Output Head
벡터 → 다음 토큰

`vocab_size`
어휘 크기 (보통 32K~150K)

`hidden_dim`
내부 벡터 차원 수

`num_layers`
Transformer Block 반복 수

`num_heads`
Multi-Head Attention 헤드 수

Qwen3.5-4B = 약 40억 개 파라미터 — hidden_dim × num_layers × 각 레이어 행렬 크기의 곱

📖 HuggingFace Transformers

🤗 Qwen3.5 Model Card

Attention 메커니즘

$$\text{Attention}(Q,K,V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) V$$

Q (Query)	검색 질의 — 현재 토큰이 "무엇을 찾고 있는가"
K (Key)	참조 키 — 각 토큰이 "나는 이런 정보야"라고 알리는 신호
V (Value)	실제 값 — Attention 가중치로 가져올 실제 정보
√d_k 스케일	dot product가 너무 커지면 softmax 기울기가 소실 → 분산 안정화

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,...,\text{head}_h)\ W^O$$

여러 관점에서 동시에 Attention 수행

직관: **각 토큰이 다른 토큰 중 어디에 주의를 기울여야 하는가**를 *학습하는 메커니즘*

📄 Attention Is All You Need (2017) 📖 HuggingFace - Attention

Auto-Regressive 생성

Step 1	[시작]	→	"에"		
Step 2	[시작]	"에"	→	"이"	
Step 3	[시작]	"에"	"이"	→	"전"
Step 4	[시작]	"에"	"이"	"전"	→ "트"

$$P(x_t \mid x_1, \ldots, x_{t-1}) = \mathrm{softmax}(W_{head} \cdot h_t)$$

h_t: 마지막 레이어 hidden state

> 매 스텝마다 전체 시퀀스를 다시 계산하면 너무 느림 → **KV-Cache**로 이전 토큰의 Key·Value를 재사용해 해결

📖 HuggingFace Generation

KV-Cache: 추론 최적화 전략

학습 (Training)

- **Teacher Forcing** — 정답 시퀀스 전체를 한 번에 입력
- 병렬 처리 가능 → GPU 효율 극대화
- KV-Cache **불필요** — 전체 컨텍스트를 한 패스로 처리
- 손실 함수: 각 위치의 다음 토큰 예측 오차 합산

추론 (Inference)

- 토큰을 **하나씩 순차적으로** 생성
- 이전 토큰의 K, V 행렬을 메모리에 저장 → 재사용
- 새 토큰은 새 Q만 계산 → 반복 연산 방지
- 긴 시퀀스일수록 KV-Cache 효과 극대화

KV 메모리: `KV memory ≈ 2 × n_layers × n_heads × d_head × seq_len × bytes`

양자화(Quantization)와 KV-Cache는 학습 알고리즘이 아니라 자원 전략이다

📖 HuggingFace KV-Cache

SFT(Supervised Fine-Tuning)란 무엇인가

대규모 사전학습(Pre-training)을 다시 하는 것이 아니라,
적은 데이터로 모델이 '어떻게 대답하는가'를 바꾸는 것

BEFORE SFT

일반적인 답변

방대한 웹 데이터로 학습된 범용 응답 패턴 — 도메인 특화 지식이 표면에 드러나지 않음

→

AFTER SFT

도메인 전문가 수준의 답변

소규모 고품질 데이터셋으로 응답 형식·어조·전문성 집중 학습 → 도메인 성능 급상승

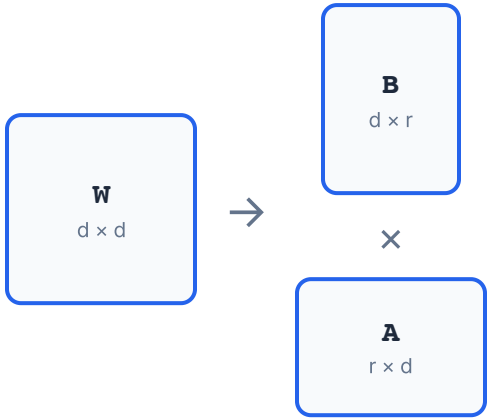
오늘 하는 것 = **도메인 성능을 빠르게 끌어올리는 실전 파인튜닝** — 사전학습 없이 소수의 예제만으로 모델 행동 교정

📖 HuggingFace TRL - SFTTrainer

📖 Qwen Fine-tuning Guide

LoRA의 수학적 원리

행렬 분해 (MATRIX DECOMPOSITION)



r = 랭크 (rank) — 매우 작은 값 (예: 16)
W는 고정, B와 A만 학습됨

수식 비교

Full FT

$$W' = W + \Delta W$$

파라미터 16,777,216개 (d=4096)

LoRA

$$\Delta W = BA, \quad r=16$$

128배 절약

파라미터 131,072개

적용

$$h = Wx + (\alpha/r)BAx$$

α = 스케일링 하이퍼파라미터

LoRA Paper (Hu et al., 2021)

HuggingFace PEFT - LoRA

LoRA vs QLoRA

항목	bf16 LoRA 이번 실습	QLoRA 참고
base weight 정밀도	bf16 (16-bit)	4-bit (NF4)
VRAM (4B 기준)	~10 GB	~6 GB
학습 정확도	손실 없음	미세 손실
디버깅	단순	양자화 오류 가능

◆ **Day-06 선택: bf16 LoRA** — Unsloth 공식 권장 방식, 안정적이고 디버깅이 단순함

QLoRA Paper (Dettmers et al., 2023)

HuggingFace PEFT - Quantization

양자화(Quantization) 메커니즘

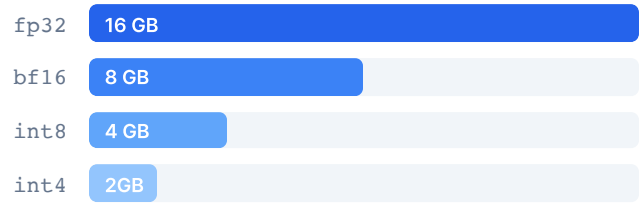
메모리 비교 (QWEN2.5-4B 기준)

정밀도	바이트/파라미터	Qwen2.5-4B
fp32	4	~16 GB
bf16	2	~8 GB
int8	1	~4 GB
int4 (NF4)	0.5	~2 GB

양자화 공식

```
x_quant = round(x / scale) + zero_point
```

메모리 절감 시각화



NF4 = 정규분포 가정 최적 양자화

이중 양자화 = scale 값도 추가 양자화

HuggingFace Quantization

bitsandbytes

Unsloth: 왜 이 도구를 쓰는가

2x
빠르게 (학습 속도)

60%
적은 VRAM 사용

핵심 API

`from_pretrained()` — 모델 로드 + 커널 최적화 자동 적용

`get_peft_model()` — LoRA adapter 부착 (rank, alpha, target modules 설정)

`for_training() / for_inference()` — 학습/추론 모드 전환 (메모리 레이아웃 최적화)

✓ **HuggingFace 100% 호환**: Unsloth 없이도 adapter 로드 가능 — 표준 PEFT 포맷 저장, HuggingFace Hub 업로드 지원

Unsloth Docs | HuggingFace PEFT | Unsloth Qwen3.5 Guide

SFT Loss: Cross-Entropy

$$L_SFT = -(1/T) \sum \log P(x_t \mid x_{<t})$$

T = 시퀀스 길이, x_t = t번째 토큰, x_<t = 이전 토큰들

1. **Label Masking**
   assistant 응답 토큰에만 loss 계산 — prompt/system 토큰은 무시

2. **Teacher Forcing**
   정답 토큰을 다음 입력으로 사용 → 한 번의 forward pass로 전체 시퀀스 학습

3. **Loss 감소의 의미**
   loss ↓ = 모델이 정답 토큰을 더 높은 확률로 예측하게 됨

⚠ **주의**: loss가 줄었는데 응답이 나쁘다면? → 데이터 품질 문제 또는 overfitting 가능성

HuggingFace TRL - SFTTrainer | HuggingFace TRL - Dataset Formats

[참고] SFT vs DPO vs RLHF

1단계 **Pretraining** → 2단계 **SFT** → 3단계 **DPO / RLHF**

방법	필요 데이터	핵심 목표
SFT	(prompt, response)	"이렇게 답해라"
DPO	(prompt, chosen, rejected)	"이 답이 저 답보다 낫다"
RLHF	rankings + reward model	"사람이 선호하는 방향으로"

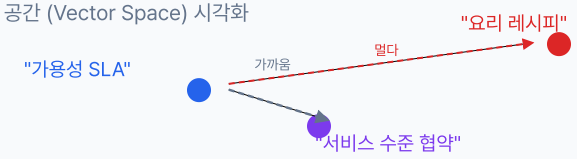
오늘은 2단계 SFT만 실습합니다 — (prompt, response) 쌍 데이터로 파인튜닝

HuggingFace TRL · DPOTrainer · RLHF Overview

임베딩의 기초 (Embedding Basics): 텍스트를 벡터로

유사한 의미 (Semantic Similarity) = 가까운 거리 (Close Distance)

벡터 공간 (Vector Space) 시각화



$$\cos(a,b) = (a \cdot b) / (||a|| \; ||b||)$$

L2 정규화 (L2 Normalization) 후 dot product = cosine similarity → 빠른 계산 (Fast Computation)
Qwen3-Embedding: 100+ 언어, 32K 컨텍스트 지원

Qwen3-Embedding-0.6B | Sentence Transformers | MTEB Leaderboard

모델 비교 (Model Comparison)

	Generative (Lab 1)	Embedding (Lab 2)
입출력	텍스트 → 텍스트	텍스트 → 벡터
목표	응답 품질	검색 품질

입력 예시 (Input Example)

"가용성 SLA (Service Level Agreement)"

↓ 임베딩 모델 (Embedding Model)

```
[0.23, -0.87, 0.41, ...]
```
1024차원 벡터 (1024-dim vector)

Embedding 모델 구조 (Model Architecture)

	Auto Encoder (BERT)	Auto Regressive (GPT/Qwen)
학습 방법	마스킹 토큰 복원 (Masked Token Recovery) — 양방향 (Bidirectional)	다음 토큰 예측 (Next Token Prediction) — 단방향 (Unidirectional)
강점	문맥 이해 (Context Understanding), 임베딩 (Embedding)	텍스트 생성 (Text Generation)

Qwen3-Embedding은 **Decoder-only** 기반이지만 임베딩 (Embedding) 용도로 특화 — 생성 모델 구조로 검색 품질 달성

CLS Pooling
BERT 방식
[CLS] 토큰 벡터 추출

Mean Pooling
주류 (Mainstream)
전체 토큰 평균

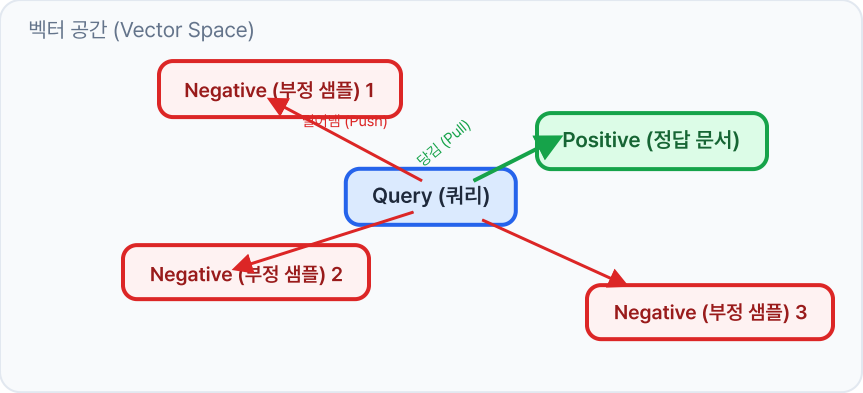
Last Token
Decoder-only 방식
마지막 토큰 벡터 사용

$$emb = (1/T) \Sigma h_t$$

Mean Pooling 수식 — T개 토큰의 히든 스테이트 (Hidden State) 평균
0.6B = 6억 파라미터 (600M Parameters). 검색 성능 (Retrieval Performance)은 크기에 반드시 비례하지 않는다

Qwen3-Embedding Model Card
Sentence Transformers Pooling

Contrastive Learning 원리 (Contrastive Learning Principle)



SFT	"이 입력에 이 출력을 생성해라 (Generate this output for this input)"
Contrastive	"이 쌍은 가깝게 (Pull closer), 저 쌍은 멀게 (Push away)"

In-batch Negatives (배치 내 부정 샘플)

`batch_size=32` → **31개** implicit negative (암묵적 부정 샘플)

`|negatives| = (batch_size - 1) + |explicit hard negatives|`

Hard Negative가 핵심 — 완전히 다른 주제 (Totally Different Topic)는 이미 구분 가능. 유사하지만 틀린 문서 (Similar but Incorrect)를 구분하는 것이 목표

Sentence Transformers Training

HuggingFace Datasets

MNRL 손실 함수 (Multiple Negatives Ranking Loss)

$$L = -(1/N) \Sigma \log [\exp(\mathrm{sim}(q, p^+) / \tau) / (\Sigma_j \exp(\mathrm{sim}(q, p_j) / \tau) + \Sigma_k \exp(\mathrm{sim}(q, n_k) / \tau))]$$

분자 (Numerator)

Query와 정답 Positive의 유사도

Query와 정답 문서 `sim(q, p+)` 의 유사도

→ 높아야 함 (Maximize)

분모 (Denominator)

모든 Passage 유사도 합

정답 + 모든 Negative 유사도 합산

→ 정답이 가장 높아야 함

τ (Temperature)

분포의 날카로움 조절 (Sharpness Control)

작을수록 → 더 날카로운 분포 (Sharper)
클수록 → 더 부드러운 분포 (Softer)

InfoNCE Loss의 변형 (Variant) — 하나의 코드 줄이 이 수식 전체를 구현 (One code line implements this entire formula)

MultipleNegativesRankingLoss

InfoNCE Paper

학습 방법의 경계 (Training Method Boundaries)

방법 (Method)	학습 데이터 (Training Data)	목표 (Objective)	비용 (Cost)
Pretraining	수조 토큰 (Trillions of Tokens)	언어 이해 기초 (Language Understanding Foundation)	매우 높음
Further Pretraining	대규모 도메인 (Large-scale Domain)	도메인 지식 주입 (Domain Knowledge Injection)	높음
SFT (Lab 1)	수백~수천 쌍 (Hundreds to Thousands)	응답 행동 조정 (Response Behavior Tuning)	낮음 Today
Embedding Tuning (Lab 2)	수백~수천 쌍 (Hundreds to Thousands)	검색 품질 개선 (Retrieval Quality Improvement)	낮음 Today

적은 데이터에서는 **SFT/ET**가 **Further Pretraining**보다 효율적 (More Efficient with Small Data)

완전한 도메인 RAG 파이프라인 (Complete Domain RAG Pipeline)

Lab 1 (SFT) + Lab 2 (ET) = RAG 완성

HuggingFace Training Guide

Unsloth Embedding Fine-tuning

핵심 메시지 (Key Messages)

1. 적은 데이터와 적은 자원에서도 도메인 특화 성능 개선 (Domain-Specific Performance Improvement)은 가능하다
2. 양자화 (Quantization)와 KV-Cache는 학습 알고리즘 (Training Algorithm)이 아니라 자원 전략 (Resource Strategy)이다
3. SFT는 생성 품질 (Generation Quality)을, Embedding Tuning은 검색 품질 (Retrieval Quality)을 개선한다
4. "가장 큰 모델 (Largest Model)"보다 "끝까지 학습하고 비교 가능한 설계 (Train to Completion with Comparable Design)"가 더 중요하다

이제 실습으로 직접 확인해 봅시다 (Let's Verify with Hands-on Practice) →

HuggingFace Transformers | HuggingFace PEFT | HuggingFace TRL | Sentence Transformers | Qwen3.5-4B | Qwen3-Embedding-0.6B | Unsloth Docs | MTEB Leaderboard